NATIONAL UNIVERSITY OF MODERN LANGUAGES
ISLAMABAD

Parallel and Distributed Computing (LAB)

Lab Report: 01

Submitted to
Sir Farhad Riaz

Submitted By
Junaid Asif
(BSAI-144)

Submission Date: October 20, 2025

Task 1

Problem Statement

Write a simple C++ program that prints “Hello World” along with integer values, representing a baseline for understanding sequential execution before moving to parallel processing.

Code

```
#include <iostream>
int main () {
   int id = 0;
   int n = 1;
   printf("hello world %d %d", id, n);
   return 0;
}
```

Output

(Add screenshot here)

Explanation

- The program runs **sequentially** using only one thread — the main thread.
- No parallel constructs or libraries are used.
- The variables id and n are manually set and printed.
- This acts as the **reference point** for understanding how parallel execution differs in performance and structure.

Task 2

Problem Statement

Use **OpenMP** to create multiple threads that print their thread ID and the total number of threads running.

Code

```
#include <iostream>
#include <stdio.h>
#include <omp.h>
int main() {
  #pragma omp parallel
```

```
  {
    printf("thread %d out of total %d\n", omp_get_thread_num(),
omp_get_num_threads());
  }
  return 0;
}
```

Output

(Add screenshot here)

Explanation

- The directive #pragma omp parallel tells the compiler to **execute the following block in parallel**.
- omp_get_thread_num() returns the **unique ID** of the currently executing thread.
- omp_get_num_threads() returns the **total number of threads** created for that parallel region.
- **Why used:**
  - To **demonstrate how OpenMP creates threads** automatically.
  - To identify **which thread executes which part** of the code.
- **Effect:**
  - The code runs simultaneously on multiple cores, improving execution speed for independent tasks.
  - Output order becomes **non-deterministic** because multiple threads print at once.

Task 3

Problem Statement

Modify the previous program to manually specify **4 threads** using OpenMP's num_threads() clause.

Code

```
#include <iostream>
#include <stdio.h>
#include <omp.h>
int main () {
  #pragma omp parallel num_threads (4)
  {
```

```
    printf("hello from thread %d\n", omp_get_thread_num());
  }
  return 0;
}
```

Output

(Add screenshot here)

Explanation

- The clause num_threads(4) explicitly sets the number of threads to **4**.
- **Why used:**
  - To show **manual thread control** instead of relying on OpenMP's default behavior.
  - Gives the programmer **predictability and control** over how resources are used.
- **Effect:**
  - Ensures consistent performance across systems with different numbers of cores.
  - Reduces unwanted CPU overhead compared to auto-threading if the workload is known.
  - Output remains non-deterministic, but limited to 4 active threads.

Task 4

Problem Statement

Increase the number of threads to **15** and observe system behavior when more threads are created than available CPU cores.

Code

```
#include <iostream>
#include <stdio.h>
#include <omp.h>
int main () {
  #pragma omp parallel num_threads (15)
  {
     printf("hello from thread %d\n", omp_get_thread_num());
  }
  return 0;
```

```
}
```

Output

(Add screenshot here)

Explanation

- This program explicitly requests **15 threads**.
- On most machines, this number will exceed the number of physical CPU cores.
- **Why used:**
  - To test **system scalability and oversubscription**.
  - To demonstrate how performance behaves when thread count > core count.
- **Effect:**
  - Causes **context switching overhead** — threads take turns sharing CPU cores.
  - Can lead to **slower performance** and jumbled output order due to excessive scheduling.
  - Helps understand **limits of parallelism** and system saturation points.

Detailed Comparative Analysis (Across All Tasks)

Aspect	Task 1	Task 2	Task 3	Task 4	Why Used / Effect on Program
Execution Type	Sequential	Parallel (Auto Threads)	Parallel (4 Threads)	Parallel (15 Threads)	Each task progressively adds more parallel control to observe effects on execution.
OpenMP Usage	❌ Not used	✅ Introduced	✅ Controlled	✅ Stressed	OpenMP is used to distribute work across threads for concurrency.
Directive Used	—	#pragma omp parallel	#pragma omp parallel num_threads(4)	#pragma omp parallel num_threads(15)	Directives control **how the compiler spawns threads** and parallelizes code.
Thread Count	1	System decides (based on cores)	Fixed at 4	Fixed at 15	Shows how thread control impacts performance and output predictability.
Function Used	printf() only	omp_get_thread_num(), omp_get_num_threads()	Same as Task 2	Same as Task 2	These OpenMP functions provide **thread-specific information**.

Aspect	Task 1	Task 2	Task 3	Task 4	Why Used / Effect on Program
Why omp_get_thread_num()	Not used (single thread)	Used to identify each thread's unique ID	Same	Same	Allows each thread to know its identity during execution.
Why omp_get_num_threads()	Not used	Used to print total threads running	Optional	Optional	Helps verify that OpenMP is actually running multiple threads.
Control Over Threads	None	Automatic (based on environment)	Manual (4 threads)	Manual (15 threads)	Manual control allows optimization and experiment with scalability.
Performance	Lowest	Improved on multi-core CPUs	Stable and efficient	Potentially degraded due to overhead	Performance improves until CPU resources are saturated.
CPU Utilization	1 core	Multiple cores	Limited to 4 cores	Over-subscribed cores	Demonstrates CPU load variation with thread count.
Output Order	Deterministic	Non-deterministic	Non-deterministic	Highly mixed	Parallelism introduces race conditions for I/O operations.
System Overhead	None	Low	Moderate	High	More threads = more scheduling overhead.
Learning Purpose	Understand serial execution	Introduce OpenMP parallelism	Apply controlled thread count	Test thread oversubscription and scaling	Each task builds upon previous to gradually teach OpenMP control, scalability, and behavior.

Commonalities Between All Tasks

- All use **C/C++** and **printf()** for output.
- All execute independently without shared data → no synchronization needed.
- Each task focuses on **thread creation and identification**, not data sharing or synchronization.
- The goal throughout is to understand **what happens as parallelism increases**.
- All demonstrate that **parallel output is non-deterministic**, meaning order of print statements can change each run.

How Each Change Affects the Program

1. **From Task 1 → Task 2:**
    - Added OpenMP → Introduced automatic multi-threading.
    - Effect: Output becomes faster and non-sequential.
2. **From Task 2 → Task 3:**

- Added num_threads(4) → Gained control over thread count.
- Effect: Predictable resource use, better optimization for fixed workloads.

3. **From Task 3 → Task 4:**
   - Increased threads to 15 → Introduced oversubscription.
   - Effect: System performance may drop, showing limits of practical parallelism.

Key Takeaways

- **OpenMP** simplifies parallel programming with minimal syntax.
- **#pragma omp parallel** creates parallel regions that divide tasks among threads.
- **Thread count control (num_threads)** is essential for optimizing performance.
- **Too many threads** cause system overhead, not speed improvement.
- Parallel programs require careful balance between **speedup** and **resource usage**.

Conclusion

Through these four tasks, we explored how OpenMP enables parallelism in C++ and how different thread configurations affect performance:

- **Task 1** established a sequential baseline.
- **Task 2** introduced automatic parallel execution.
- **Task 3** demonstrated explicit thread control.
- **Task 4** examined system behavior under excessive threading.

This lab solidifies the foundational understanding of **thread creation, management, and control in OpenMP**, setting the stage for more advanced topics like **synchronization, reduction, and data-sharing models** in future labs.